



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

PROMETHEUS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Kubernetes

TOTAL: 10 QUESTIONS

Q1 What problem does Prometheus solve in DevOps or infrastructure?

Threat Model

Q6 How does Prometheus integrate with CI/CD pipelines?

Observability

Q2 What is Prometheus's core architecture?

Architecture

Q7 How do you debug a failed Prometheus deployment?

Lifecycle

Q3 How does Prometheus define infrastructure or configuration as code?

Configuration

Q8 What are security best practices for Prometheus?

Compliance

Q4 How does Prometheus ensure idempotency?

Hardening

Q9 How does Prometheus scale for large environments?

Testing

Q5 How do you handle secrets in Prometheus?

SSO / IdP

Q10 What are common mistakes teams make when adopting Prometheus?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Prometheus addresses a specific concern

Mitigation

Prometheus addresses a specific concern—provisioning, configuration management, orchestration, or CI/CD—by automating manual steps that are error-prone, slow, or not reproducible when done by hand.

A6 CI/CD pipelines call Prometheus in automated

Observability

CI/CD pipelines call Prometheus in automated steps: plan on a pull request to preview changes and apply on merge to main. Approval gates can require a human to review the plan before changes go to production.

A2 Prometheus has a control plane that

Primitives

Prometheus has a control plane that stores desired state and a data plane (agents or push-based SSH) that enforces it. Understanding this distinction clarifies where configuration is evaluated and where changes are applied.

A7 Check Prometheus's logs for the specific

Automation

Check Prometheus's logs for the specific error message and the resource that failed. For infrastructure tools, re-run with increased verbosity (-v, --debug). Review the state file or event history to understand what changed before the failure.

A3 Prometheus uses declarative files (YAML, HCL,

Hardening

Prometheus uses declarative files (YAML, HCL, or a DSL) to express desired state. A plan/diff phase shows what will change before applying. Version-controlling these files enables peer review and rollback.

A8 Rotate credentials used by Prometheus, restrict

Governance

Rotate credentials used by Prometheus, restrict network access to its control plane, enable audit logging of all operations, and use least-privilege service accounts. Never run Prometheus with admin credentials in production.

A4 Prometheus operations are designed to produce

Gotchas

Prometheus operations are designed to produce the same result when run multiple times. Applying the same config twice converges to the desired state rather than creating duplicate resources. This makes automation safe to re-run.

A9 Large Prometheus deployments use

Assessment

Large Prometheus deployments use workspaces or namespaces to isolate environments, remote state backends for team collaboration, and modular code to avoid a single monolithic config that is slow to plan/apply.

A5 Secrets should never be stored in

Federation

Secrets should never be stored in plain-text in Prometheus config files. Use a secrets backend (Vault, AWS Secrets Manager) and inject values at runtime via environment variables or encrypted vaults supported by the tool.

A10 Common pitfalls include storing secrets in

Optimization

Common pitfalls include storing secrets in plain-text config, skipping the plan step before apply, not reviewing state drift, and not having a rollback strategy when a deployment fails partway through.